PRÁCTICA N.º 10 — HOJA DE EVIDENCIAS

Observabilidad: métricas, registros y trazas

COM-600 · Microservicios · Gestión 2/2026 · Tema 8 — Observabilidad y Monitoreo

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ingeniería de sistemas
Fecha de entrega	23/10/2026
Sistema operativo, memoria RAM disponible y versiones de Docker y de Node	Windows 10, 8gb ram, Docker version 29.7.2, build a7dcaa6, node version v24.18.1
Repositorio (URL)	

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

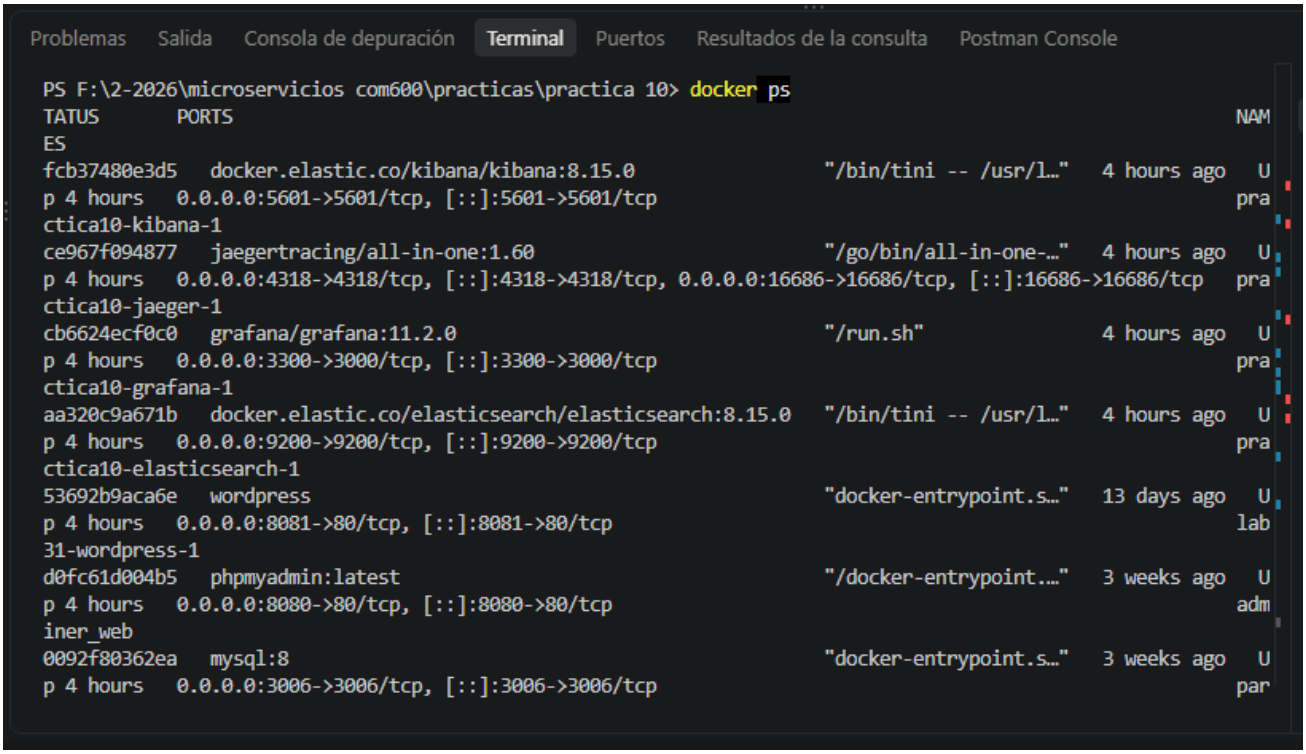
Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Entorno: levantar la pila de observabilidad	1, 2	1	X
Laboratorio 1 — Registros con sentido: de console.log al JSON estructurado	3, 4	2	X
Laboratorio 2 — Registros centralizados: los dos servicios en un solo lugar	5, 6, 7	3	X
Laboratorio 3 — Métricas: exponer, recolectar y consultar	8, 9, 10	4	X
Laboratorio 4 — Tablero en Grafana: peticiones, latencia y errores	11, 12, 13	5	X
Laboratorio 5 — Trazado distribuido con OpenTelemetry	14, 15, 16	6	X
Laboratorio 6 — Correlación de los tres pilares	17, 18	7	X
Laboratorio 7 — Alertas y objetivos de nivel de servicio	19, 20, 21	8	

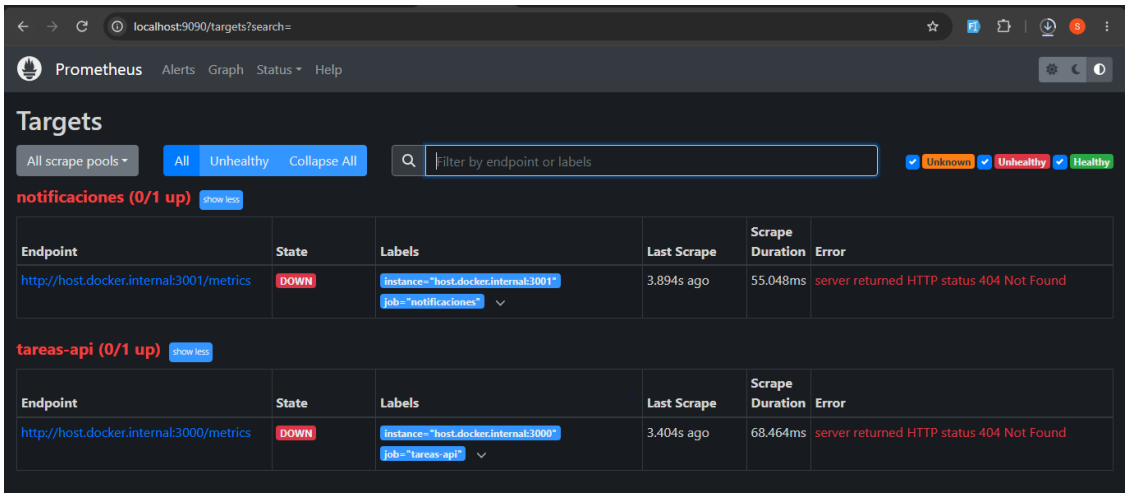
PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

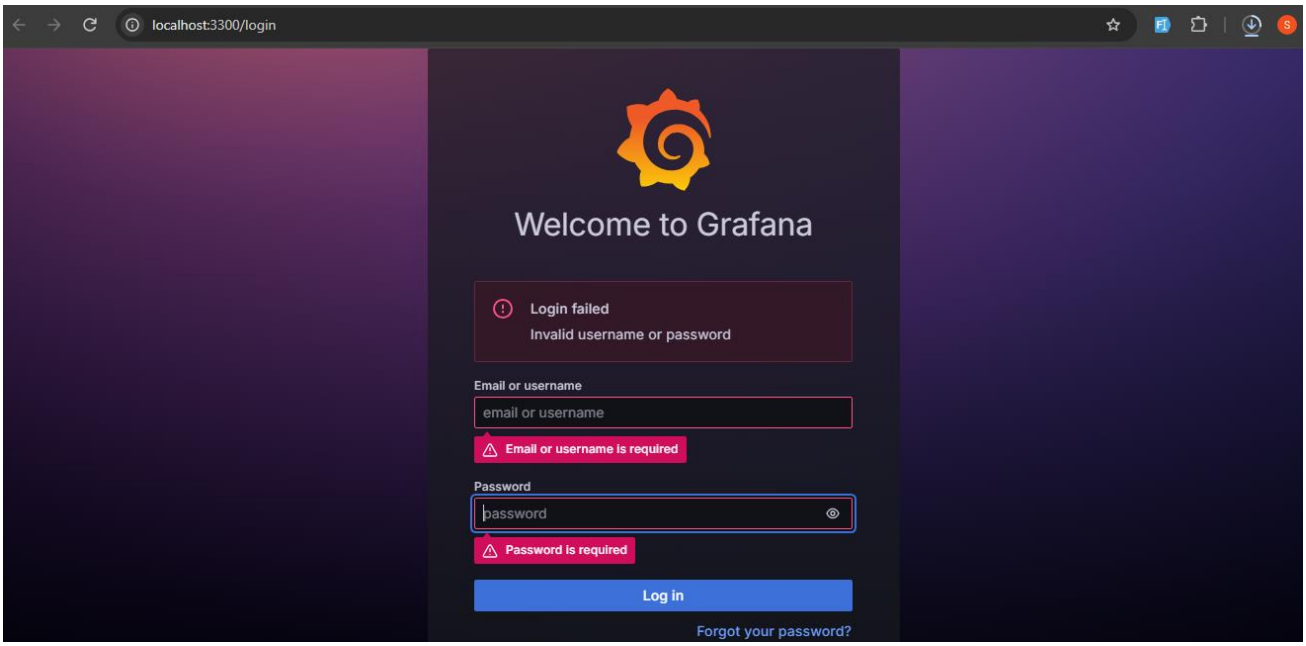
Laboratorio 0 — Entorno: levantar la pila de observabilidad

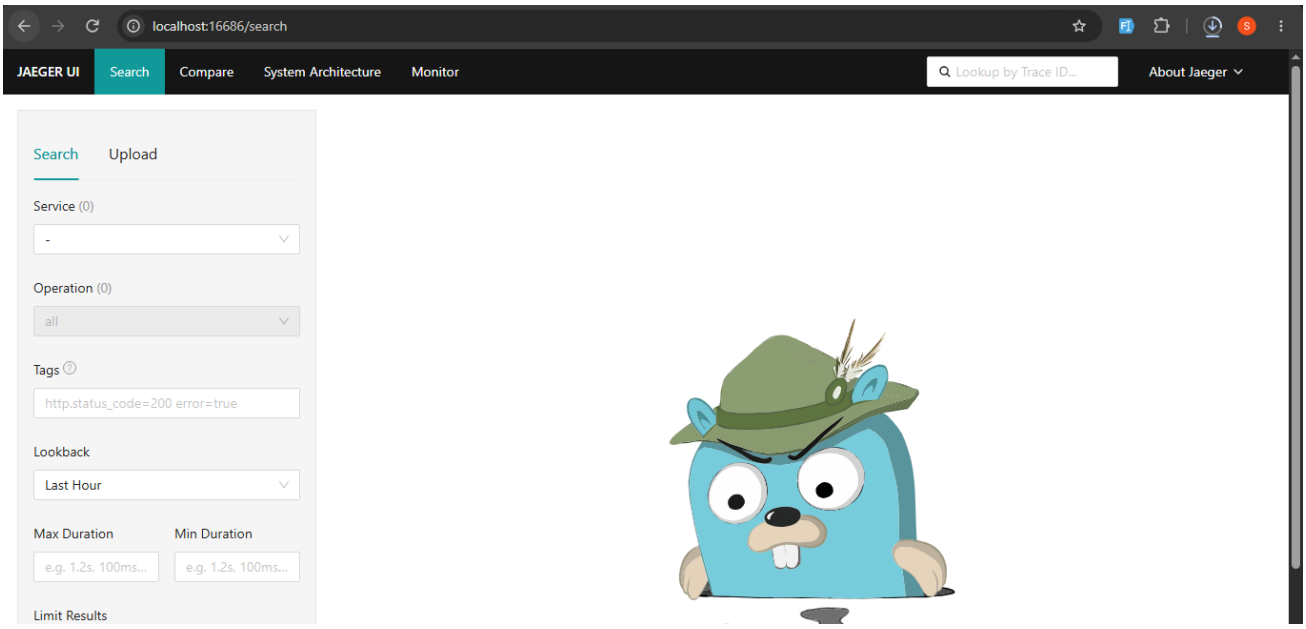
Captura N.º 1: salida de docker compose ps con los seis contenedores de la pila de observabilidad en ejecución.

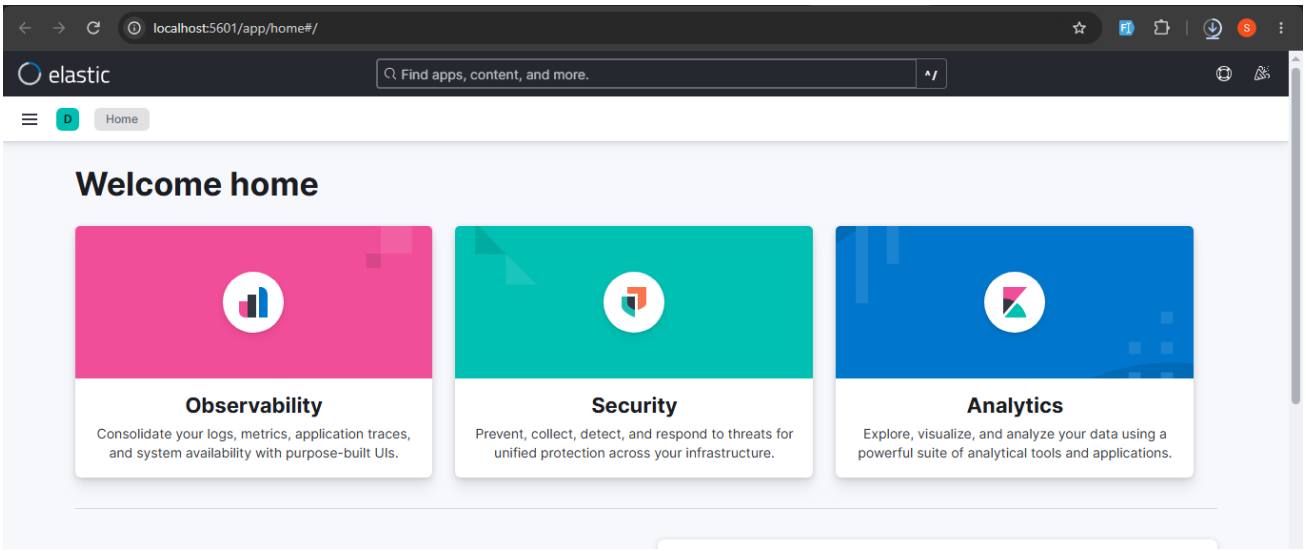


Captura N.º 2: las cuatro interfaces abiertas: la página de objetivos de Prometheus, la pantalla inicial de Grafana, la de Jaeger y la de Kibana.




localhost:3300/login
Welcome to Grafana
Login failed
Invalid username or password
Email or username
email or username
Email or username is required
Password
password
Password is required
Log in
Forgot your password?


localhost:16686/search
JAEGER UI
Search
Compare
System Architecture
Monitor
Lookup by Trace ID...
About Jaeger
Search
Upload
Service (0)
-
Operation (0)
all
Tags
http.status_code=200 error=true
Lookback
Last Hour
Max Duration
Min Duration
e.g. 1.2s, 100ms...
e.g. 1.2s, 100ms...
Limit Results



Pregunta N.º 1. Ninguno de los cuatro instrumentos que acaba de levantar corre dentro de sus servicios: todos están al lado. ¿Qué gana el equipo con esa separación y qué obliga a hacer a cambio en el código de los servicios?

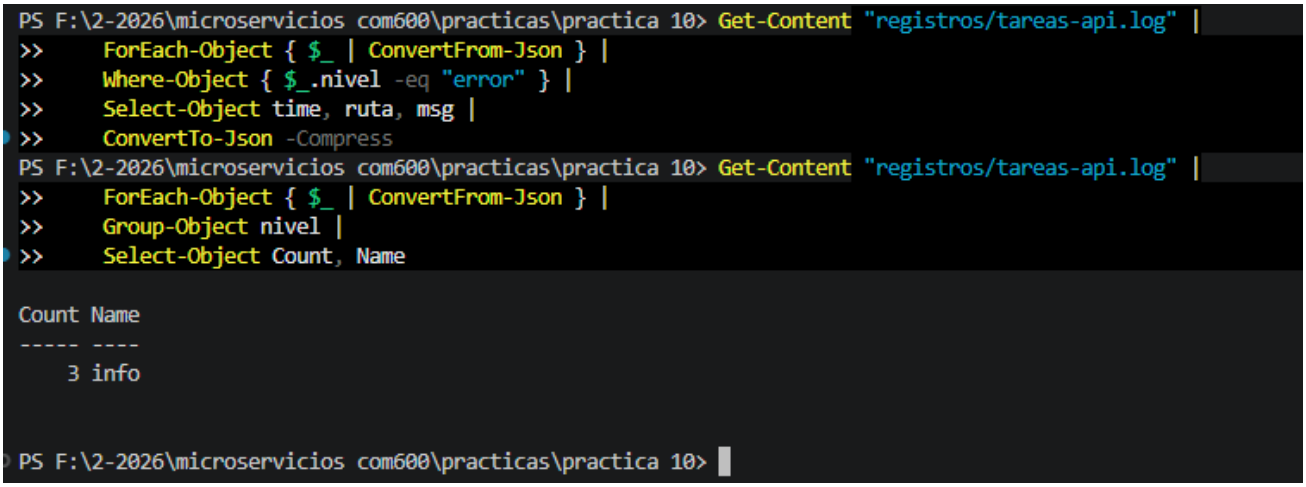
- **Qué se gana:** Desacoplamiento total y resiliencia. Si las herramientas de monitoreo fallan, se saturan o se reinician, los microservicios siguen atendiendo a los usuarios sin degradar su rendimiento ni caerse.
- **Qué obliga a cambio en el código:** Obliga a exponer datos estandarizados hacia afuera (rutas `/metrics`, envío de logs a disco/stdout y exportación de trazas por red vía OTLP).

Laboratorio 1 — Registros con sentido: de console.log al JSON estructurado

Captura N.º 3: las líneas del archivo registros/tareas-api.log en formato JSON, donde se lean el nivel, la marca de tiempo y el idCorrelacion.

```
PS F:\2-2026\microservicios com600\practicas\practica 10> curl.exe -s http://localhost:3000/tareas
[{"id":1,"titulo":"Instrumentar el servicio","completada":false,"estado":"PENDIENTE"}]
PS F:\2-2026\microservicios com600\practicas\practica 10> Get-Content registros/tareas-api.log -Tail 5
{"nivel":"info","time":"2026-10-24T01:30:11.268Z","servicio":"tareas-api","entorno":"laboratorio","idCorrelacion":"c34ab67a-3209-4c57-8eef-5f37bbedfbef","ruta":"/tareas","metodo":"GET","cantidad":0,"msg":"Listado de tareas consultado"}
{"nivel":"info","time":"2026-10-24T01:30:32.462Z","servicio":"tareas-api","entorno":"laboratorio","idCorrelacion":"cf0c98a8-0db6-4383-af88-9f56de57a633","ruta":"/tareas","metodo":"POST","tareaId":1,"titulo":"Instrumentar el servicio","msg":"Tarea creada"}
{"nivel":"info","time":"2026-10-24T01:31:59.740Z","servicio":"tareas-api","entorno":"laboratorio","idCorrelacion":"4ba74eed-1594-4f7b-92ed-cc436b7b6903","ruta":"/tareas","metodo":"GET","cantidad":1,"msg":"Listado de tareas consultado"}
PS F:\2-2026\microservicios com600\practicas\practica 10>
```

Captura N.º 4: la salida de los dos comandos con jq: los errores filtrados y el conteo por nivel.



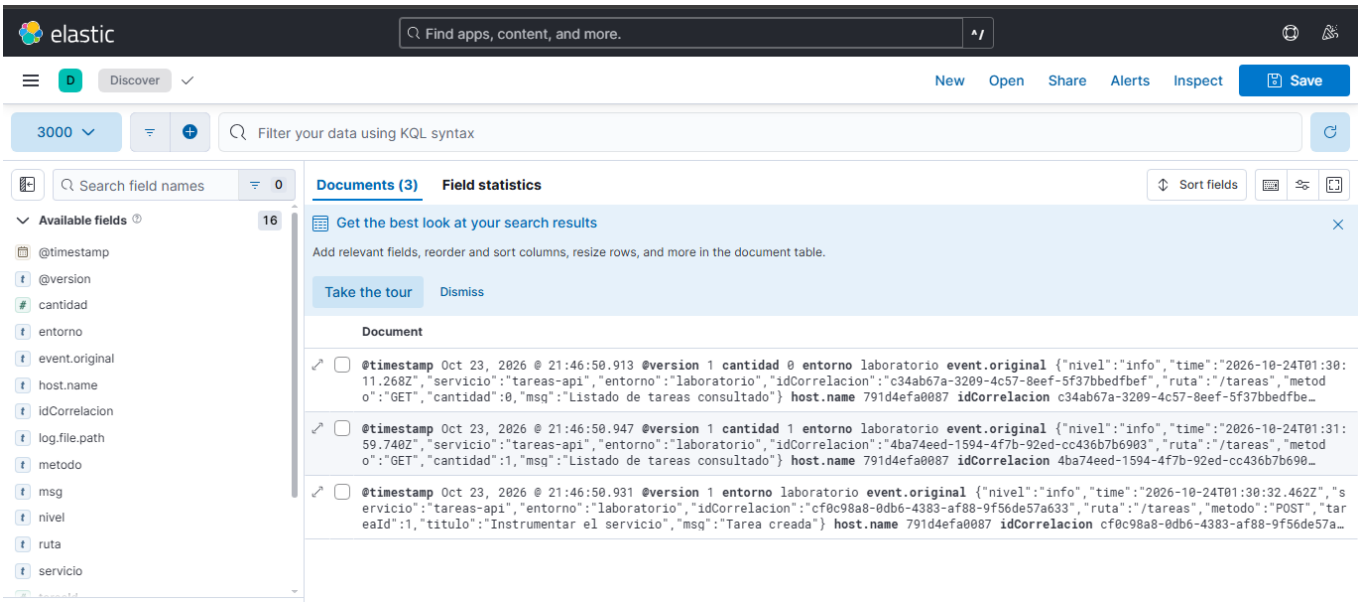
Pregunta N.º 2. Con el registro anterior, en texto libre, esas dos consultas exigían leer las líneas una por una o inventar una expresión regular frágil. Explique qué cambió exactamente para que ahora sean posibles, y por qué eso importa más cuando hay diez servicios que cuando hay uno.

- **Qué cambió:** Se pasó de texto plano no estructurado a JSON con claves fijas (`nivel`, `servicio`, `idCorrelacion`, etc.). Esto permite a los analizadores filtrar y agregar por propiedades exactas en vez de interpretar texto con expresiones regulares.
- **Por qué importa más con diez servicios:** Con un solo servicio es posible leer una terminal; con diez servicios se generan miles de líneas por segundo en formatos distintos. El JSON unificado permite centralizar, filtrar por servicio o gravedad y correlacionar eventos sin ambigüedades.

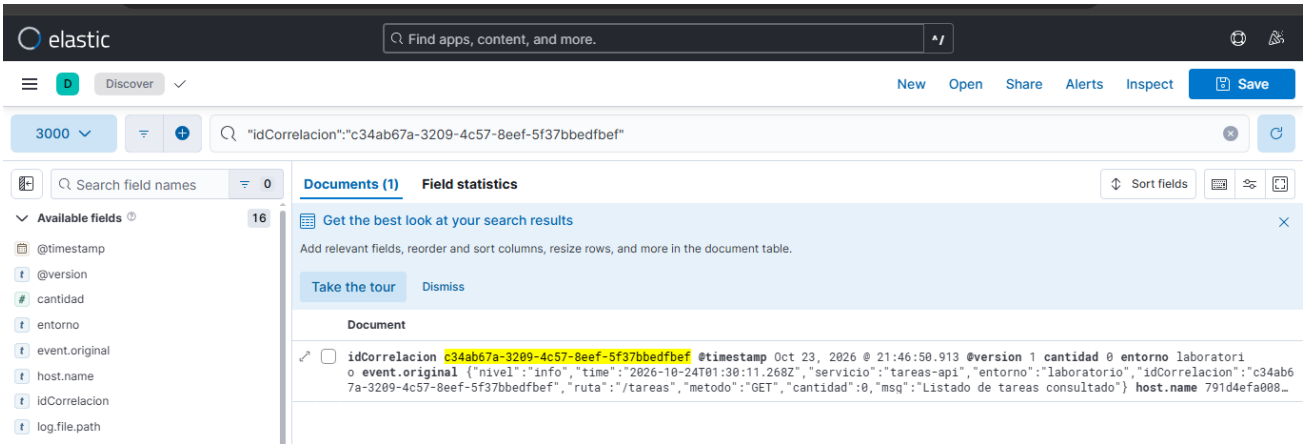
Laboratorio 2 — Registros centralizados: los dos servicios en un solo lugar

Captura N.º 5: la vista Discover de Kibana mostrando registros de los dos servicios en la misma lista, con la columna servicio visible.





Captura N.º 6: la consulta por idCorrelacion en Kibana devolviendo los registros de los dos servicios ordenados por hora.



Captura N.º 7: la consulta por idCorrelacion devolviendo únicamente registros de tareas-api, sin los de notificaciones.

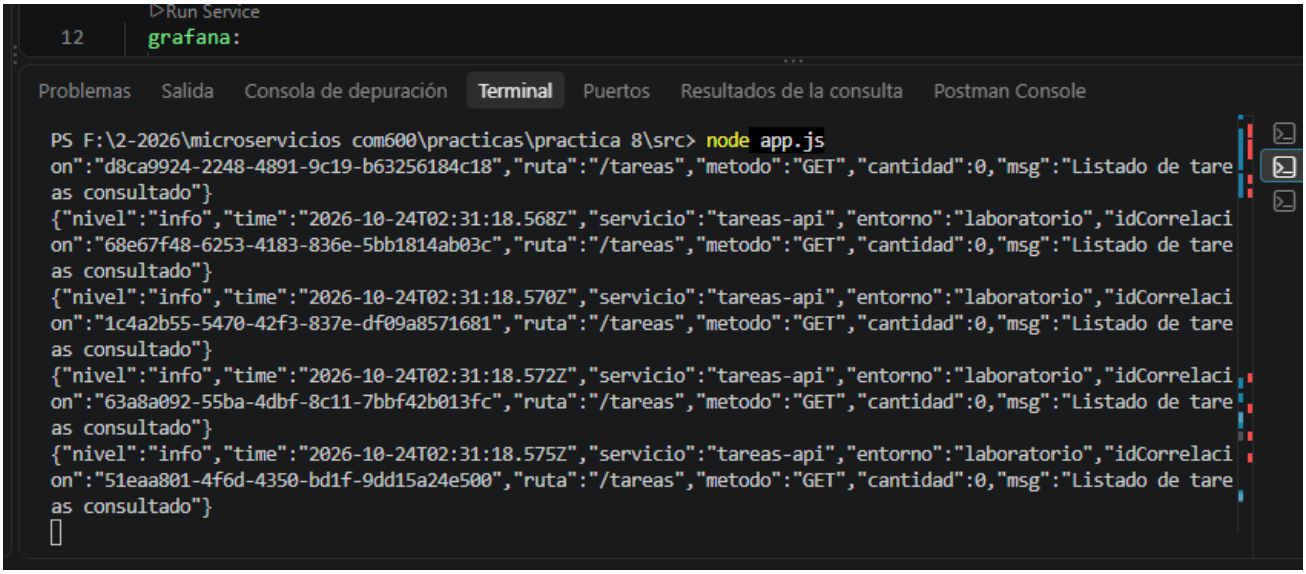


Pregunta N.º 3. El identificador de correlación no lo genera ninguna herramienta: lo genera y lo propaga el código de sus servicios. ¿Qué consecuencia tiene eso para un equipo con quince servicios escritos por gente distinta, y qué mecanismo propondría usted para que nadie pueda olvidarse de propagarlo?

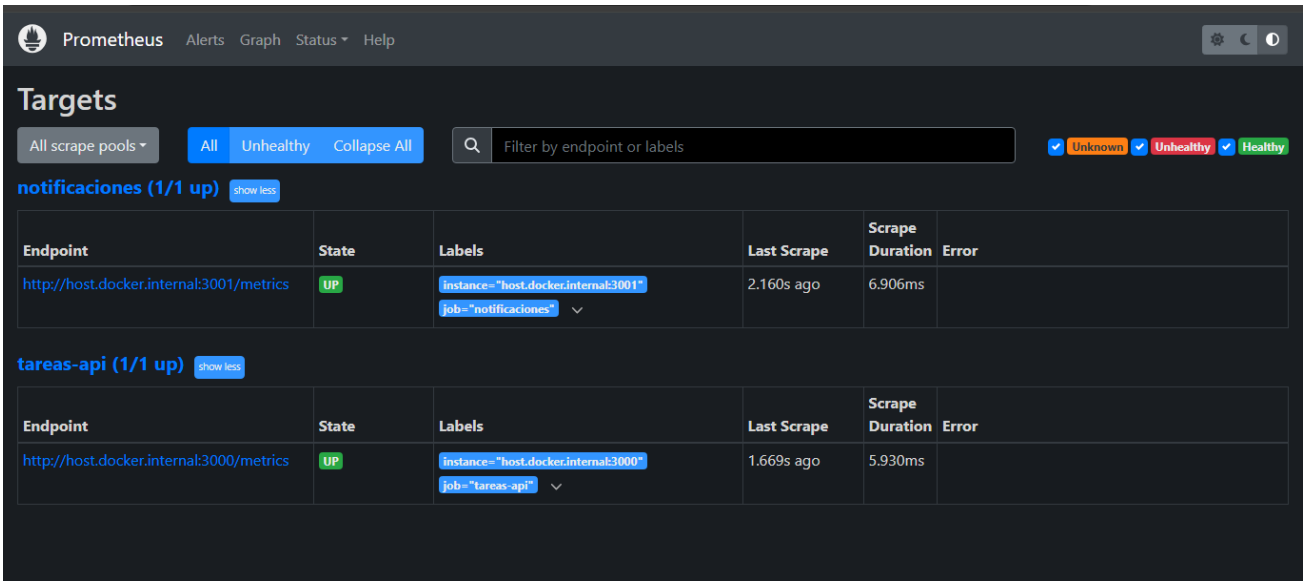
- **Consecuencia:** Si un desarrollador olvida reenviar la cabecera en una llamada saliente, la cadena de trazabilidad se rompe y los registros posteriores quedan huérfanos e imposibles de correlacionar.
- **Mecanismo propuesto:** Usar una librería o cliente HTTP base compartido en la organización (o instrumentación automática mediante OpenTelemetry) que intercepte todas las peticiones salientes e inyecte automáticamente la cabecera `x-correlation-id` sin intervención manual del desarrollador.

Laboratorio 3 — Métricas: exponer, recolectar y consultar

Captura N.º 8: la salida de curl sobre /metrics mostrando las series http_request_duration_seconds con sus buckets, su suma y su conteo.



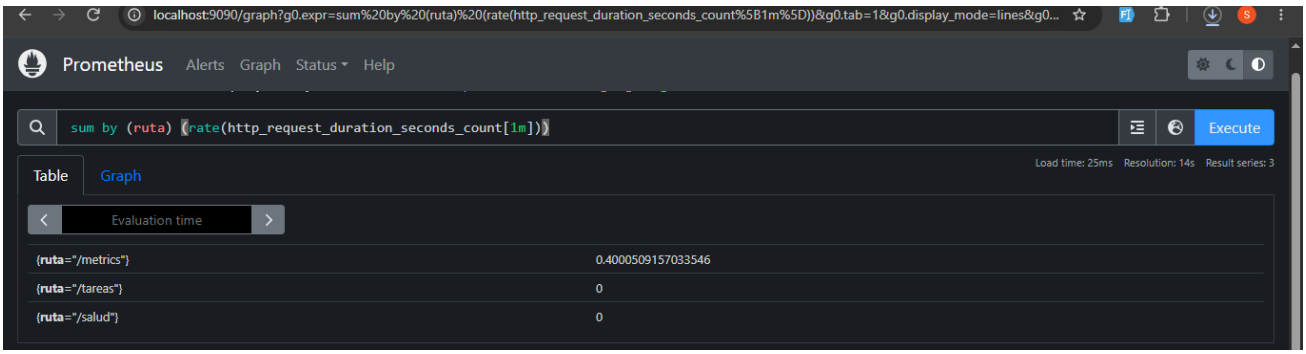
Captura N.º 9: la página de objetivos de Prometheus con tareas-api y notificaciones en estado UP y su última recolección exitosa.



Pregunta N.º 4. Prometheus va y pide el número cada cinco segundos, en lugar de que el servicio se lo empuje cuando quiere. Nombre una ventaja concreta y una desventaja concreta de ese modelo, pensando en un servicio que se reinicia veinte veces al día por despliegues.

- **Ventaja (modelo pull):** Prometheus controla el ritmo de recolección y evita que los servicios saturen la base de datos de monitoreo si hay picos de tráfico; además detecta de inmediato si un servicio murió porque el objetivo pasa a `DOWN`.
- **Desventaja:** Si un servicio se reinicia constantemente, puede morir y renacer entre dos intervalos de consulta (en menos de 5 segundos), perdiéndose las métricas acumuladas en memoria antes del reinicio.

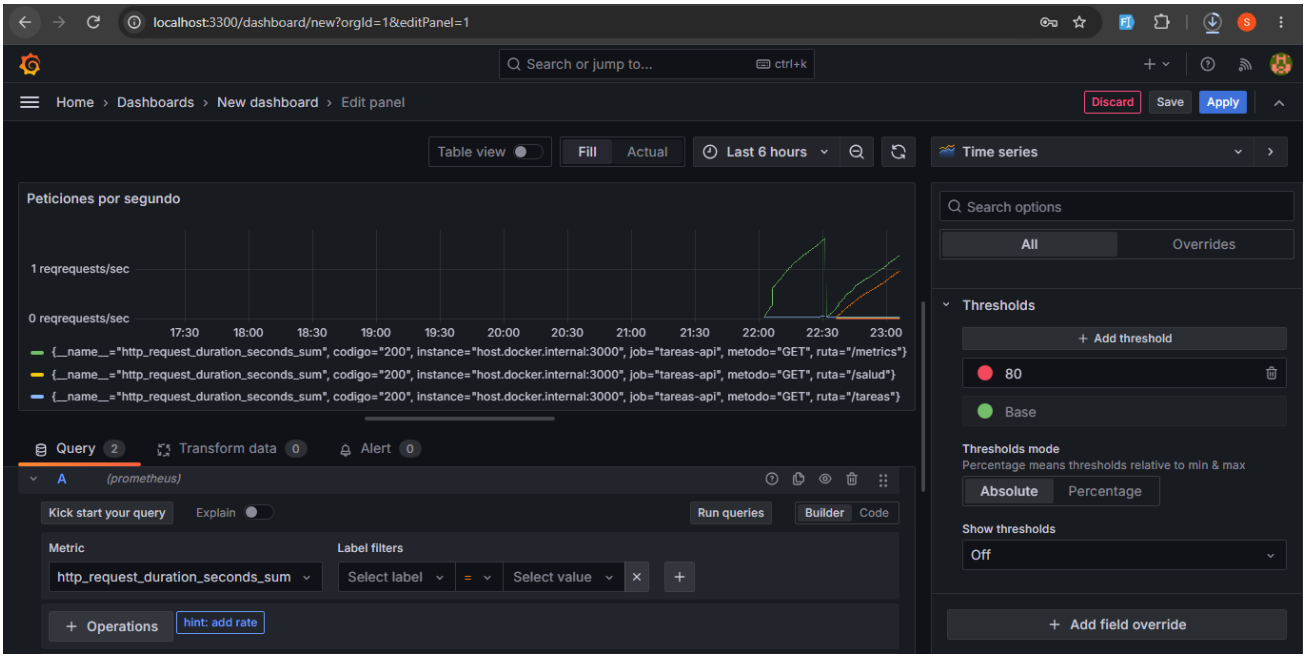
Captura N.º 10: el gráfico de la primera consulta PromQL en Prometheus, con la curva de peticiones por segundo separada por ruta.



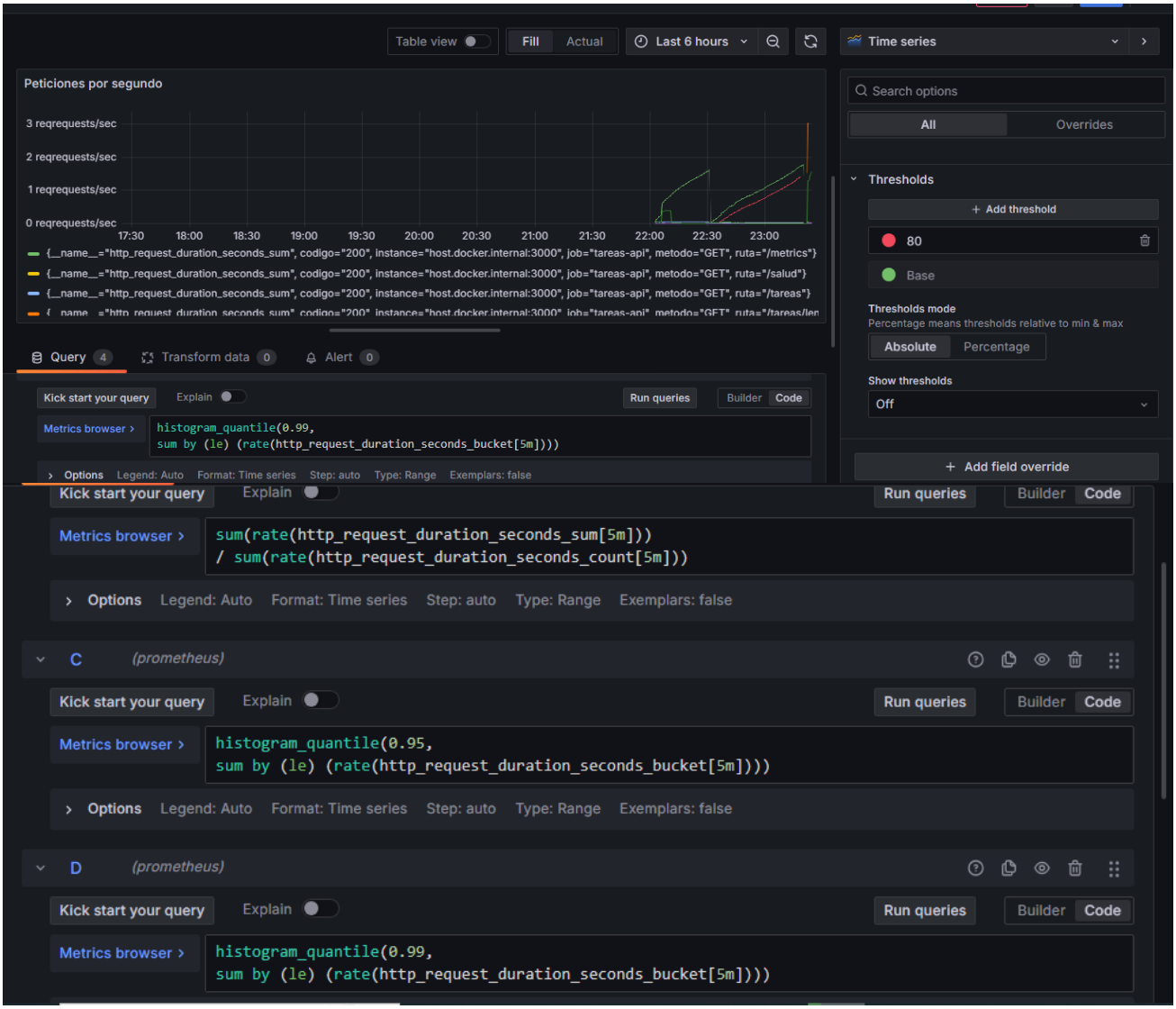


Laboratorio 4 — Tablero en Grafana: peticiones, latencia y errores

Captura N.º 11: el panel de peticiones por segundo en Grafana con la curva moviéndose y la leyenda por ruta.



Captura N.º 12: el panel de latencia con las tres curvas superpuestas, donde se vea el percentil 99 despegado del promedio.



Pregunta N.º 5. Con 19 peticiones de 20 ms y una de 1500 ms, el promedio queda cerca de 94 ms y el percentil 95 se dispara. Si su acuerdo de servicio dice «responder en menos de 300 ms», ¿cuál de las dos cifras usaría para decidir si lo está cumpliendo y por qué? Explique a quién representa cada número.

- **Cifra a usar:** El percentil 95 (p95).
- **Por qué:** Porque el promedio diluye el problema y miente (94 ms parece cumplir el acuerdo de 300 ms, cuando en realidad hay usuarios esperando 1500 ms).
- **A quién representa cada número:**
    - El **promedio** representa un valor matemático abstracto que no refleja la vivencia de ningún usuario real.

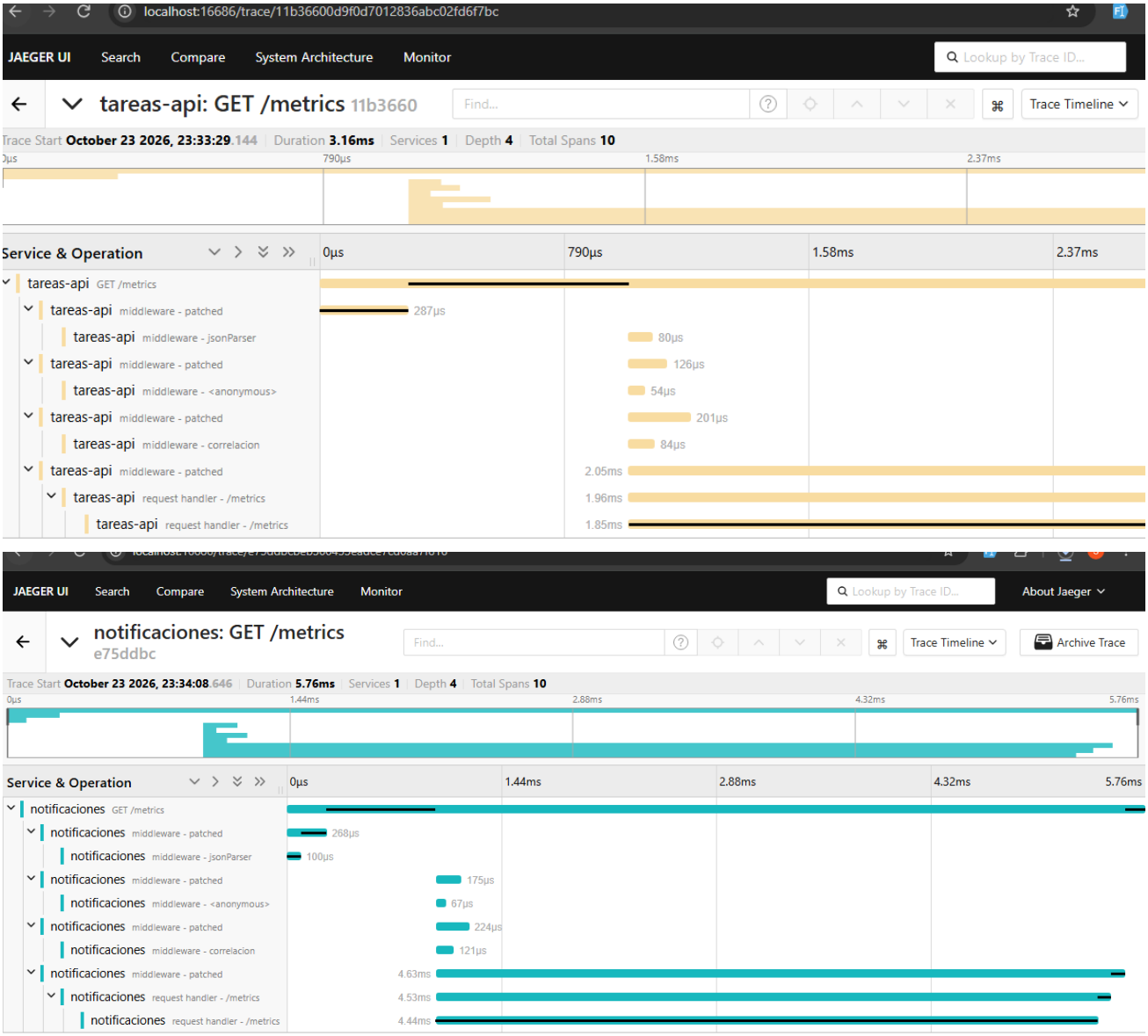
- El **percentil 95** representa al 95 % de los usuarios que tuvieron una respuesta rápida y visibiliza al 5 % más afectado que sufrió una degradación inaceptable.

Captura N.º 13: el tablero completo con los tres paneles (peticiones por segundo, latencia con percentiles y tasa de errores) y la tasa de errores subiendo.

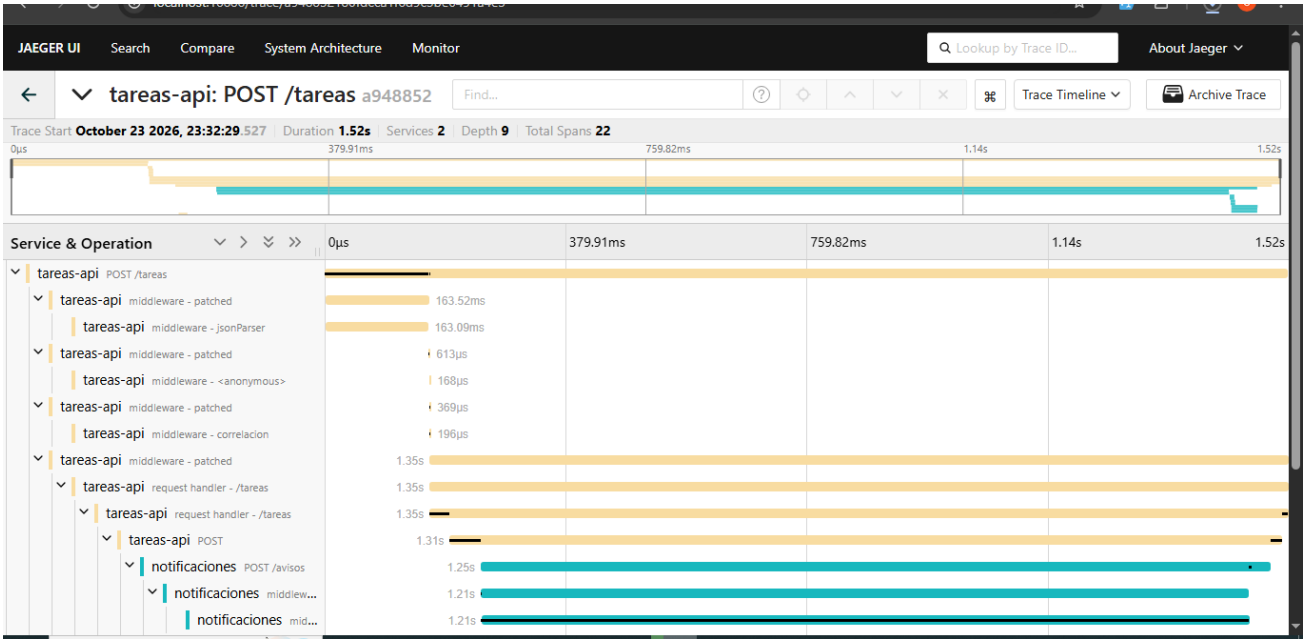


Laboratorio 5 — Trazado distribuido con OpenTelemetry

Captura N.º 14: la cascada completa de la traza en Jaeger, con los spans de tareas-api y de notificaciones en la misma petición y la duración de cada uno.



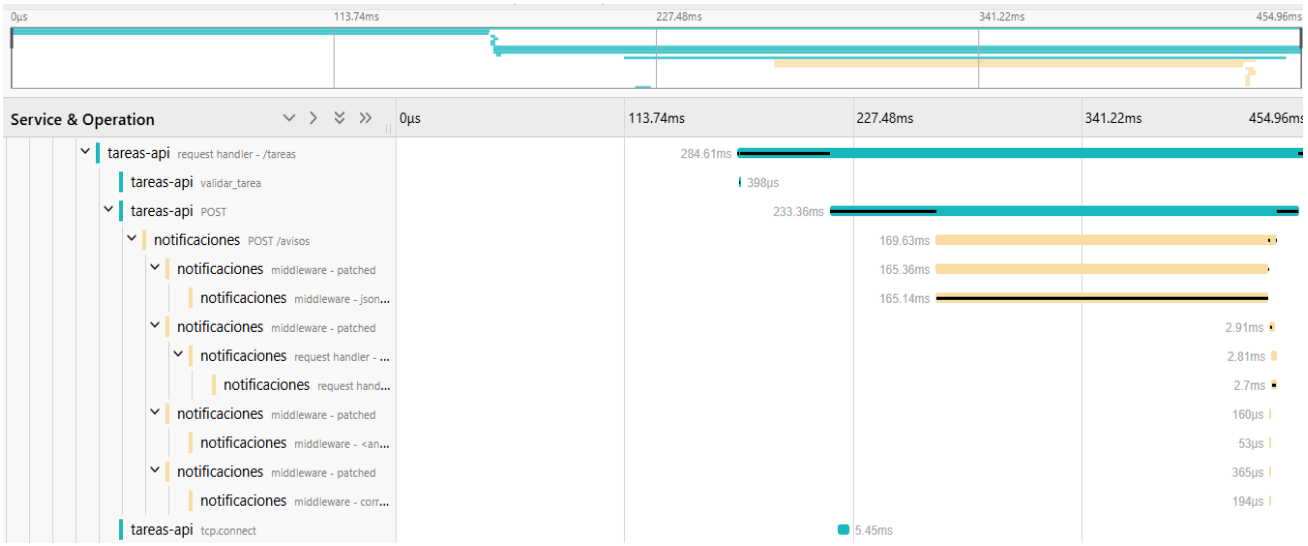
Captura N.º 15: la traza lenta en Jaeger con el span de notificaciones ocupando casi todo el ancho, y su duración visible.



Pregunta N.º 6. El servicio de tareas responde en 1500 ms y sus propios registros no muestran ningún error. Sin trazas, ¿qué habría tenido que hacer para localizar la causa? Compare ese trabajo con lo que acaba de hacer y estime cuánto tiempo le habría llevado con ocho servicios en lugar de dos.

- **Qué habría tenido que hacer:** Revisar manualmente los logs de cada servicio, contrastar marcas de tiempo entre máquinas con relojes desincronizados y adivinar en qué punto se originó la demora.
- **Comparación:** Con trazas se ve en un segundo el span culpable con su duración exacta; sin trazas, en un sistema de ocho servicios, rastrear una petición demorada puede tomar varias horas de análisis forense en lugar de unos pocos clics.

Captura N.º 16: la cascada en Jaeger con el span propio validar_tarea visible y sus atributos desplegados en el panel inferior.

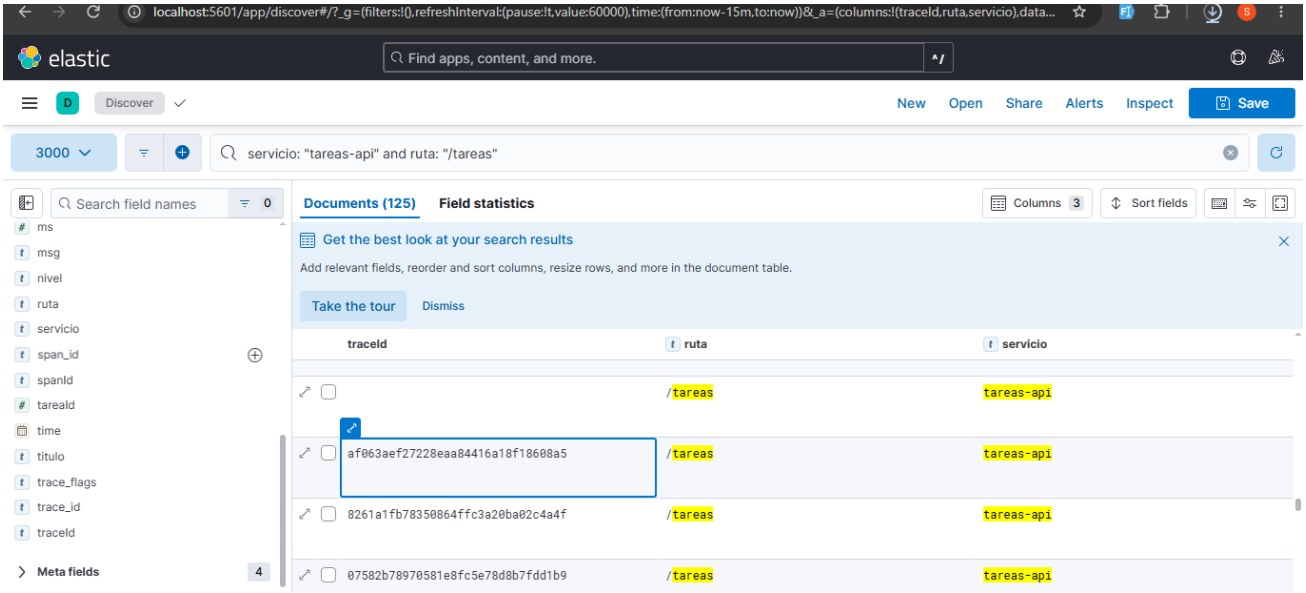


Laboratorio 6 — Correlación de los tres pilares

Captura N.º 17: el pico de latencia en Grafana con la hora visible y, al lado, la consulta de Kibana acotada a ese minuto mostrando el campo traceId.



Captura N.º 18: la traza abierta en Jaeger a partir del traceId copiado del registro, mostrando la misma petición.



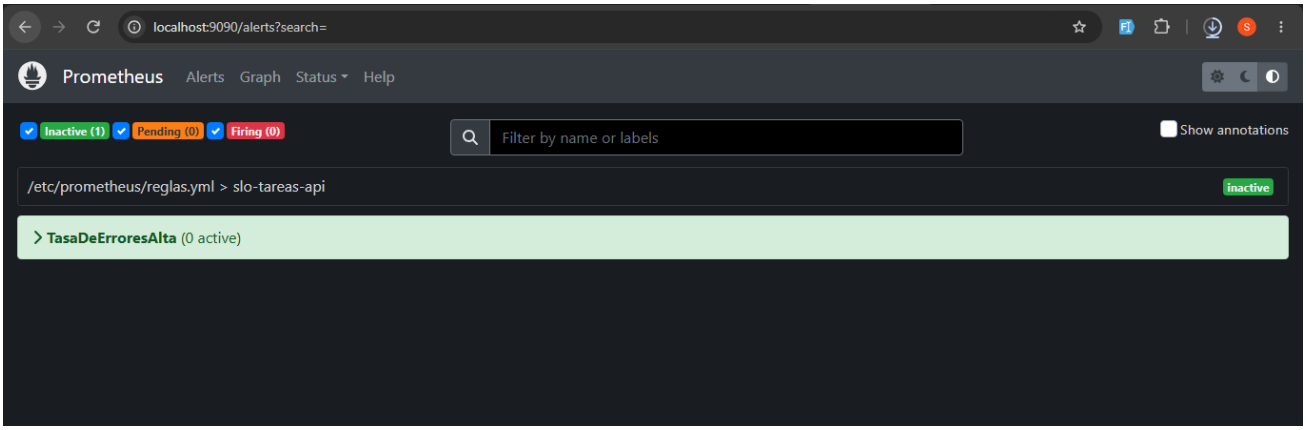
Pregunta N.º 7. Describa con sus palabras qué pregunta responde cada uno de los tres pilares y por qué ninguno de los tres puede reemplazar a los otros dos. Dé un ejemplo de una falla que la métrica NO detectaría y otro de una falla que la traza NO explicaría.

- **Qué responde cada pilar:**
  - **Métricas:** *¿Hay un problema y cuándo ocurre?* (avisa anomalías a nivel global).
  - **Registros:** *¿Qué detalle ocurrió en ese momento?* (contexto del error y mensajes de negocio).
  - **Trazas:** *¿Dónde ocurrió y qué camino siguió?* (muestra el tramo o dependencia causante en la red).

- **Falla que la métrica NO detectaría:** Un error en los datos de respuesta donde el código HTTP sea 200 pero el cuerpo traiga información vacía o errónea para un usuario específico.
- **Falla que la traza NO explicaría:** Una saturación gradual del disco o fuga de memoria global en el servidor que no corresponde a una sola petición en particular.

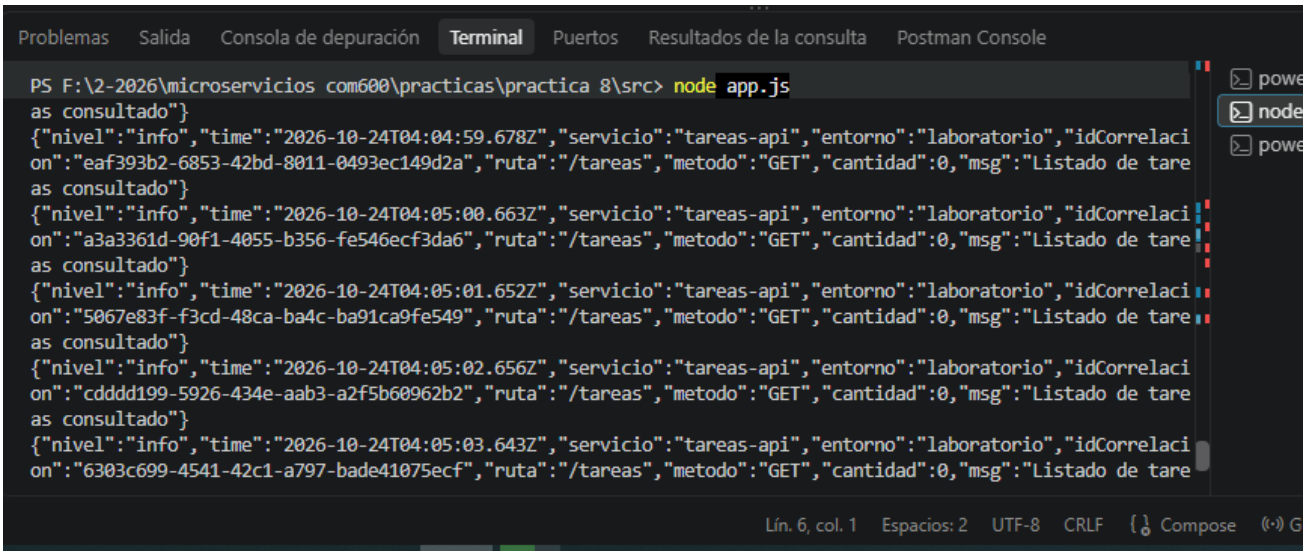
Laboratorio 7 — Alertas y objetivos de nivel de servicio

Captura N.º 19: la página de alertas de Prometheus con la regla TasaDeErroresAlta cargada en estado inactive.

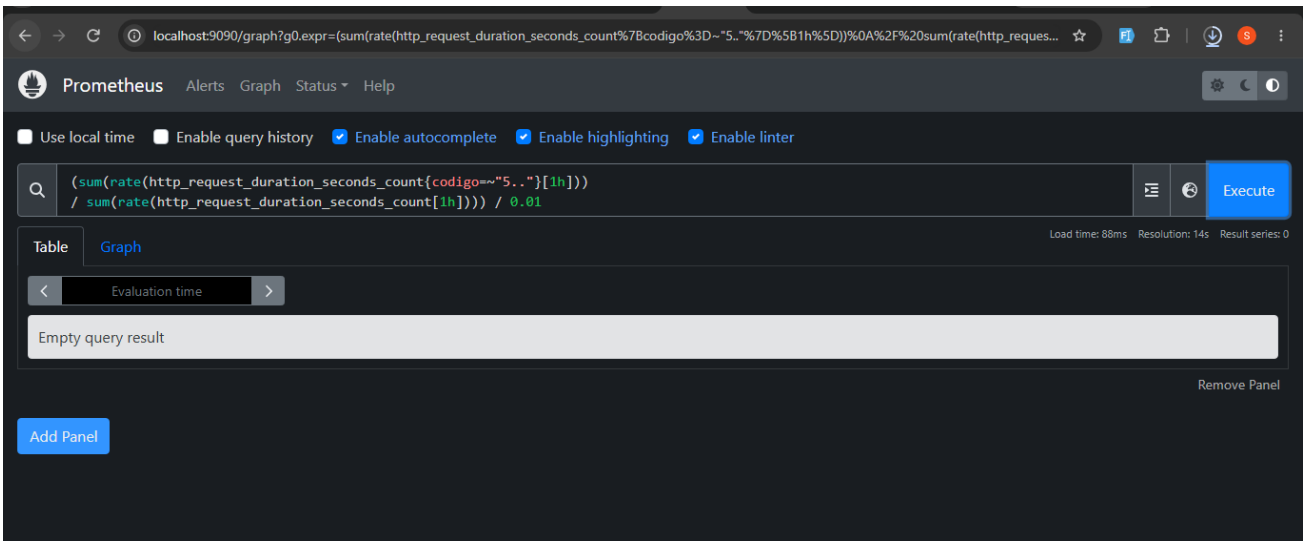


Captura N.º 20: la alerta TasaDeErroresAlta en estado FIRING en Prometheus, con la etiqueta de severidad y el resumen visibles.





Captura N.º 21: la consulta del presupuesto de error consumido en Prometheus, con el valor resultante visible.



Pregunta N.º 8. De estas tres alertas posibles, ¿cuál merece despertar a una persona a las tres de la mañana y cuáles no? (a) el uso de CPU de un contenedor superó el 80 %; (b) más del 5 % de las peticiones de pago falla hace cinco minutos; (c) un contenedor se reinició una vez. Justifique cada caso en términos de lo que el usuario percibe y de lo que la persona de guardia puede hacer al respecto en ese momento.

- **(a) Uso de CPU superó el 80 %: No despertar.** El usuario no percibe lentitud si las peticiones responden a tiempo. Puede ser un pico transitorio normal y se puede revisar al día siguiente.
- **(b) Más del 5 % de las peticiones de pago falla hace 5 minutos: Sí merece despertar.** Afecta directamente al usuario y al negocio (pérdida de dinero y compras bloqueadas). La guardia debe intervenir de inmediato para revertir el despliegue o mitigar la falla.

- **(c) Un contenedor se reinició una vez: No despertar.** Docker o Kubernetes lo levantan solos de manera transparente; el usuario apenas lo nota. Solo amerita alarma si entra en un bucle continuo de reinicios (*CrashLoopBackOff*).

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

El manejo de grafana y Jaeger

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

Acomodar la sangrías de los archivos .yaml acomodando cuantos espacios debe tener cada línea

¿Qué haría distinto si tuviera que repetir esta práctica?

Revisar la documentación de grafana, prometheus, jeager, etc.

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

_____ _____

Firma del estudiante Fecha